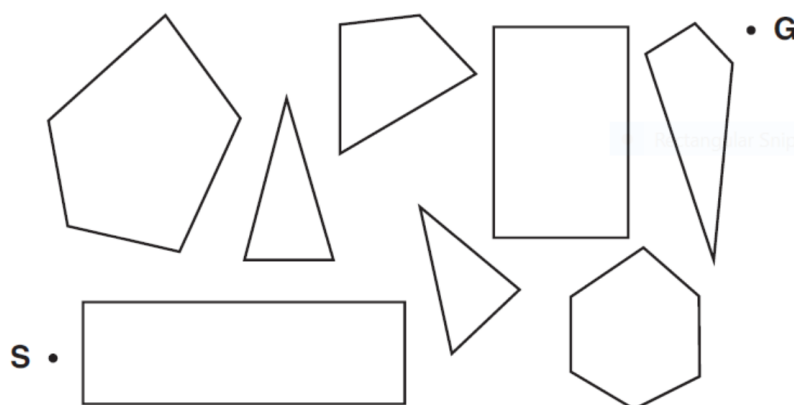


Bài Tập Thực Hành AI Tuần 4

Võ Minh Thịnh
MSSV: 22280087

1 Bài toán

Xét bài toán tìm đường đi ngắn nhất từ điểm S đến điểm G trong một mặt phẳng có các chướng ngại vật là các đa giác lồi như hình



Thuật toán xác định các điểm có thể nhìn thấy từ một điểm

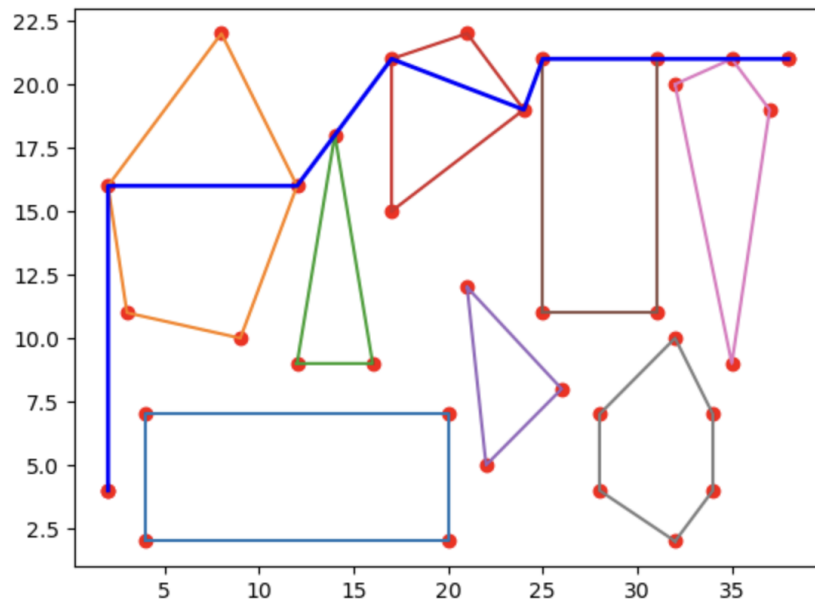
Gọi P là một đỉnh đang xét, V là tập cạnh của tất cả các đa giác. Với mỗi cạnh đa giác, gọi là AB , trong tập V :

1. Tạo d_1 từ P và A , d_2 từ P và B , d_3 từ A và B .
2. Xét tất cả các đỉnh Q còn lại với d_1 , d_2 , d_3 :
 - Nếu $d_1(Q) \cdot d_1(B) > 0$ và $d_2(Q) \cdot d_2(A) > 0$ và $d_3(Q) \cdot d_3(P) < 0$, thì Q là đỉnh **không** nhìn thấy được từ P .
 - Ngược lại, Q là đỉnh nhìn thấy được từ P .

2 Cài đặt chương trình

2.1 Vấn đề

Trong chương trình, đường đi được tạo ra không hợp lệ khi đã tạo ra cạnh chéo mới trong đa giác và đi theo đường đó. Vì đây vẫn là các đỉnh có thể nhìn thấy được theo như thuật toán phía trên. Nên ta cần chỉnh sửa lại chương trình



Hình 1: Một đường đi không hợp lệ của thuật toán GBFS

2.2 Sửa lỗi và bổ sung

Sửa lỗi thực dòng trong hàm can_see

```
def can_see(self, start):
    see_list = list()
    cant_see_list = list()

    for polygon in self.polygons:
        for edge in self.polygons[polygon]:
            for point in self.get_points():
                if start == point:
                    cant_see_list.append(point)
                if start in self.get_polygon_points(polygon):
                    for poly_point in self.get_polygon_points(polygon):
                        if poly_point not in self.get_adjacent_points(start):
                            cant_see_list.append(poly_point)
                if point not in cant_see_list:
                    if start.can_see(point, edge):
                        if point not in see_list:
                            see_list.append(point)
                        elif point in see_list:
                            see_list.remove(point)
                            cant_see_list.append(point)
                        else:
                            cant_see_list.append(point)
    return see_list
```

Hình 2: Hàm can_see trước khi sửa

```
1. def can_see(self, start): # hàm trả về các điểm mà điểm start có thể nhìn thấy được
2.     see_list = list()
3.     cant_see_list = list()
4.
5.     for polygon in self.polygons:
6.         for edge in self.polygons[polygon]:
7.             for point in self.get_points():
8.                 if start == point:
9.                     cant_see_list.append(point)
10.                if start in self.get_polygon_points(polygon):
11.                    for poly_point in self.get_polygon_points(polygon):
12.                        if poly_point not in self.get_adjacent_points(start):
13.                            cant_see_list.append(poly_point)
14.                if point not in cant_see_list:
15.                    if start.can_see(point, edge):
16.                        if point not in see_list:
17.                            see_list.append(point)
18.                        elif point in see_list: # sửa lỗi thực dòng
19.                            see_list.remove(point)
20.                            cant_see_list.append(point)
21.                        else: # sửa lỗi thực dòng
22.                            cant_see_list.append(point)
23.    return see_list # sửa lại lỗi thực đầu dòng
```

Hình 3: Hàm can_see sau khi sửa

Bổ sung hàm `reachable_points` để xác định node có thể di chuyển đến (hợp lệ) từ node đang xét (trong `see_list`)

Khi xét các node có thể nhìn thấy từ một node cụ thể, ta có các trường hợp sau:

1. **Node có thể nhìn thấy nằm ở một đa giác khác:** Trong trường hợp này, node đang xét có thể nhìn thấy một node thuộc đa giác khác, nghĩa là giữa chúng không có bất kỳ chướng ngại vật nào.
2. **Node có thể nhìn thấy nằm trong cùng một đa giác với node đang xét:**
 - (a) **Trường hợp 2.1: Có cạnh nối giữa node đang xét và node có thể nhìn thấy:** Khi đó, hai node này đã được nối trực tiếp với nhau bằng một cạnh, và do đó có thể di chuyển đến nhau một cách hợp lệ.
 - (b) **Trường hợp 2.2: Không có cạnh nối giữa node đang xét và node có thể nhìn thấy:** Đây là trường hợp không hợp lệ, vì để đến được node đó, ta cần phải vẽ một đường chéo mới trong đa giác, điều này làm cho đường đi không hợp lệ.

```
1. def reachable_points(self, start):
2.     reachable_points = set()
3.     polygons = self.get_polygons()
4.     can_see = self.can_see(start)
5.     for point in can_see:
6.         if point.polygon_id != start.polygon_id:
7.             reachable_points.add(point)
8.         else:
9.             # kiểm tra xem có phải là node liền kề không
10.            if point in self.get_adjacent_points(start):
11.                reachable_points.add(point)
12.            # kiểm tra xem có phải là node không kề có thể đi đến không (không tạo ra đư
13.            edge = Edge(start, point)
14.            if edge in polygons[start.polygon_id] or edge2 in polygons[start.polygon_id]
15.                reachable_points.add(point)
16.     return reachable_points
```

Hình 4: Hàm bổ sung `reachable_points`

Cập nhật lại hàm search

Thay vì duyệt các node trong `can_see` thì sẽ thay thành duyệt các node trong `reachable_points`

```
1. def search(graph, start, goal, func): # hàm tìm kiếm đường đi dựa theo hàm heuristic
2.     closed = set()
3.     queue = PriorityQueue()
4.     queue.put((0 + func(graph, start), start))
5.     if start not in closed:
6.         closed.add(start)
7.     while not queue.empty():
8.         cost, node = queue.get()
9.         if node == goal:
10.            return node
11.        for i in graph.reachable_points(node): # duyệt qua tất cả các điểm có thể đi tới từ
12.            new_cost = node.g + euclid_distance(node, i)
13.            if i not in closed or new_cost < i.g:
14.                closed.add(i)
15.                i.g = new_cost
16.                i.pre = node
17.                new_cost = func(graph, i)
18.                queue.put((new_cost, i))
19.    return node
```

Hình 5: Hàm search sau khi cập nhật

2.3 Cài đặt các thuật toán BFS, DFS, UCS

BFS (Breadth - First - Search)

```
1. def bfs(graph, start, goal): # hàm tìm kiếm đường đi dựa theo thuật toán BFS
2.     visited = set()
3.     queue = Queue()
4.     queue.put(start)
5.     if start not in visited: # nếu điểm start chưa được thăm thì thêm vào tập hợp visited
6.         visited.add(start)
7.     while not queue.empty():
8.         node = queue.get()
9.         if node == goal:
10.            return node
11.        for i in graph.reachable_points(node): # Duyệt qua tất cả các điểm có thể đi tới từ
12.            if i not in visited:
13.                visited.add(i)
14.                i.pre = node # Lưu điểm trước đó để hỗ trợ truy vết đường đi.
15.                queue.put(i) # Thêm điểm này vào hàng đợi để tiếp tục xử lý ở các bước sau
```

Hình 6: Hàm tìm kiếm đường đi dựa theo thuật toán BFS

DFS (Depth - First - Search)

```
1. def dfs(graph, start, goal): # hàm tìm kiếm đường đi dựa theo thuật toán DFS
2.     visited = set()
3.     stack = []
4.     stack.append(start)
5.     if start not in visited:
6.         visited.add(start)
7.     while stack:
8.         node = stack.pop()
9.         if node == goal:
10.            return node
11.        for i in graph.reachable_points(node): # Duyệt qua tất cả các điểm có thể đi tới từ
12.            if i not in visited:
13.                visited.add(i)
14.                i.pre = node # Lưu lại điểm trước đó để có thể truy vết đường đi
15.                stack.append(i) # Đưa điểm này vào stack để tiếp tục thăm
```

Hình 7: Hàm tìm kiếm đường đi dựa theo thuật toán DFS

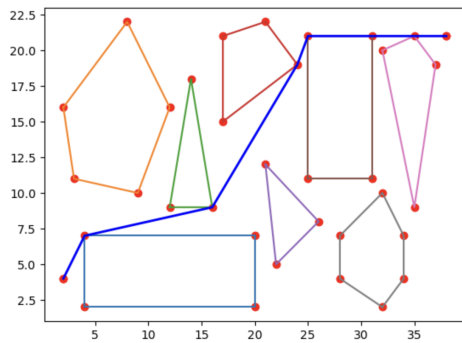
UCS (Uniform Cost - Search)

```
1. def ucs(graph, start, goal): # hàm tìm kiếm đường đi dựa theo thuật toán UCS
2.     visited = set()
3.     queue = PriorityQueue() # hàng đợi ưu tiên
4.     queue.put((0, start))
5.     if start not in visited:
6.         visited.add(start)
7.     while not queue.empty():
8.         cost, node = queue.get() # lấy ra node có chi phí nhỏ nhất
9.         if node == goal:
10.            return node
11.        for i in graph.reachable_points(node): # duyệt qua tất cả các điểm có thể đi tới từ
12.            new_cost = node.g + euclid_distance(node, i)
13.
14.            if i not in visited or new_cost < i.g: # khi i được thăm thì thêm i vào đường
15.                # khi i đã được thăm nhưng chi phí mới nhỏ hơn thì cập nhật lại chi phí
16.                visited.add(i)
17.                i.g = new_cost
18.                i.pre = node
19.                queue.put((new_cost, i))
20.     return node
```

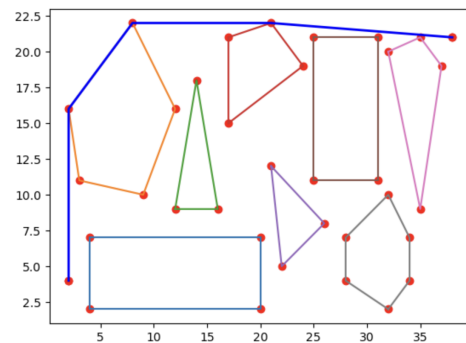
Hình 8: Hàm tìm kiếm đường đi dựa theo thuật toán UCS

3 Thực thi chương trình

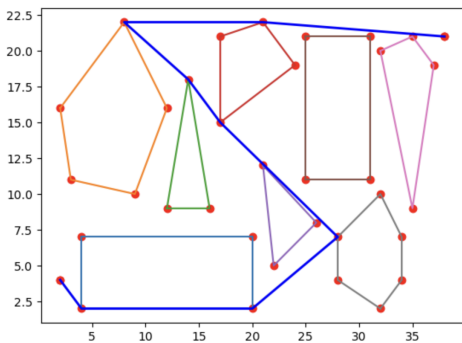
3.1 Kết quả thực thi chương trình (Graph)



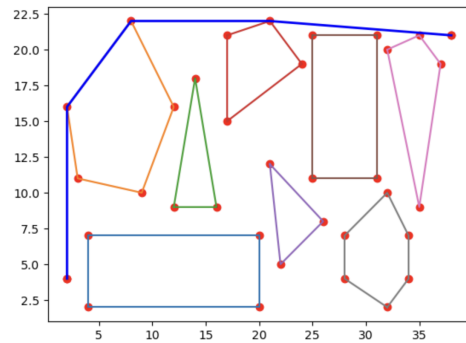
Hình 9: Kết quả của A* Search



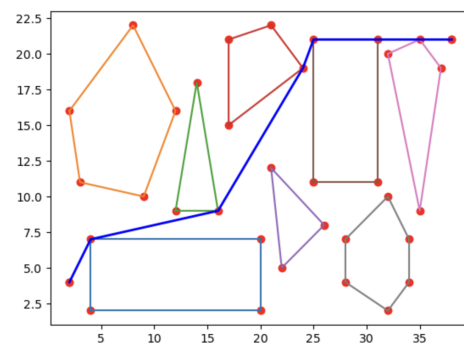
Hình 10: Kết quả của GBFS



Hình 11: Kết quả của DFS



Hình 12: Kết quả của BFS



Hình 13: Kết quả của UCS

3.2 Đường đi theo tọa độ điểm và chi phí

A* Search

```
Đang thăm: (38, 21)
[(2, 4), -1] -> [(4, 7), 0] -> [(16, 9), 2] -> [(24, 19), 3] -> [(25, 21), 5] -> [(31, 21), 5] -> [(35, 21), 6] -> [(38, 21), -1]
Tổng chi phí đường đi: 43.814
```

Greedy Best First Search

```
Đang thăm: (38, 21)
[(2, 4), -1] -> [(2, 16), 1] -> [(8, 22), 1] -> [(21, 22), 3] -> [(38, 21), -1]
Tổng chi phí đường đi: 58.513999999999996
```

UCS (Uniform Cost - Search)

```
/var/folders/sz/tr8rp49x6n5szdzt_59v5p/40000gn/1/ipykernel_20634/54286/1.py:102: DeprecationWarning: NotImplemented should not be used
return list(filter(None.__ne__, [edge.get_adjacent(point) for edge in self.edges]))
[(2, 4), -1] -> [(4, 7), 0] -> [(16, 9), 2] -> [(24, 19), 3] -> [(25, 21), 5] -> [(31, 21), 5] -> [(35, 21), 6] -> [(38, 21), -1]
Tổng chi phí đường đi: 43.814
```

BFS (Breadth - First - Search)

```
return list(filter(None.__ne__, [edge.get_adjacent(point) for edge in self.edges]))
[(2, 4), -1] -> [(2, 16), 1] -> [(8, 22), 1] -> [(21, 22), 3] -> [(38, 21), -1]
Tổng chi phí đường đi: 58.513999999999996
```

DFS (Depth - First - Search)

```
return list(filter(None.__ne__, [edge.get_adjacent(point) for edge in self.edges]))
[(2, 4), -1] -> [(4, 2), 0] -> [(20, 2), 0] -> [(28, 7), 7] -> [(17, 15), 3] -> [(14, 18), 2] -> [(8, 22), 1] -> [(21, 22), 3] -> [(38, 21), -1]
Tổng chi phí đường đi: 83.346
```

4 Nhận xét

Các thuật toán tìm đường có hiệu quả và cách tiếp cận khác nhau:

- **A*** nổi bật nhất khi cân bằng giữa chi phí thực tế và dự đoán (heuristic), cho kết quả tối ưu với chi phí thấp.
- **Greedy Best-First Search** tập trung vào heuristic nên nhanh nhưng dễ đi sai hướng nếu dự đoán kém.
- **Uniform Cost Search** đảm bảo tối ưu khi không có heuristic nhưng có thể tốn nhiều thời gian hơn.
- **Breadth-First Search** và **Depth-First Search** không tối ưu, thường mở rộng không cần thiết, thích hợp với bài toán đơn giản hoặc không yêu cầu tối ưu chi phí.